A

Synopsis

on

OFFLINE SEMANTIC SEARCH BROWSER EXTENSION

in partial fulfillment of the requirement for the degree

of

Bachelor of Technology

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Varun Attri (2401331690064)
Tanmay Singh (2401331690056)
Md. Sami Khan (2401331690036)

Under the supervision of

Ms. Pooja Kumari

(Designation CSE)

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY

GREATER NOIDA

Index

Supervisor Sign:

1. INTRODUCTION

In the modern digital era, a vast amount of information is generated and stored in the form of webpages, documents, PDFs, and other digital content. Efficiently searching and retrieving relevant information from this data has become a critical requirement for users. Most commonly used search systems rely on keyword-based searching, where results are displayed only when exact or similar words are found in the content.

However, keyword-based search systems have significant limitations. They do not understand the actual meaning or context of the user's query. As a result, users often fail to find relevant information when different words or expressions are used for the same concept. For example, a search for *"car engine problem"* may not return results containing *"vehicle motor issue"*, even though both have similar meanings. This reduces the accuracy and efficiency of traditional search methods.

To overcome these limitations, **semantic search** has emerged as an advanced approach. Semantic search focuses on understanding the meaning and context of words rather than just matching keywords. It uses Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques to analyze the relationship between words and provide more accurate results. Semantic search systems convert text into numerical representations called embeddings, which capture the meaning of the text and allow comparison based on similarity.

Although modern semantic search systems provide better accuracy, they are mostly cloud-based and depend on internet connectivity. These systems require large AI models, high computational power, and often involve sending user data to remote servers. This leads to several issues such as increased cost, higher response time due to network delay, and concerns related to data privacy and security.

In addition, common browser tools like Ctrl + F are limited to simple keyword matching and cannot perform meaning-based search. They are not efficient for searching large documents or understanding complex queries. This creates a gap between basic search tools and advanced AI-based systems.

The proposed project aims to bridge this gap by developing a **High-Speed Offline Semantic Search Browser Extension**. This system is designed to perform semantic search directly within the browser without requiring any internet connection. It uses a lightweight AI embedding model that runs locally on the user's device, making the system fast, efficient, and privacy-friendly.

The core idea of the system is to convert both the document content and the user query into vector embeddings and then compare them using similarity measures. The system extracts text from webpages, PDFs, and documents, processes it, and stores the embeddings locally. When a user enters a query, it is also converted into an embedding and compared with stored data to find the most relevant results.

1.1 Objectives

The main objectives of this project are:

- To replace traditional keyword-based search with semantic meaning-based search.
- To achieve fast response time (less than 1 second) for real-time usability.
- To design a system using a lightweight AI model suitable for browser environments.
- To enable semantic search across webpages, PDF files, and text documents.
- To allow users to search across multiple documents simultaneously.
- To ensure the system works completely offline without internet dependency.
- To implement efficient caching and indexing mechanisms for faster repeated searches.
- To develop a privacy-friendly system where user data remains local.

2. EXISTING SYSTEMS

The existing search systems primarily rely on traditional keyword-based search techniques. These systems function by matching the exact words or phrases entered by the user with the content available in documents or webpages. While this approach is simple and widely used, it has several limitations in understanding the actual intent behind a query.

Most conventional search engines and browser search tools do not consider the context or semantic meaning of the text. As a result, they fail to provide accurate results when different words with similar meanings are used. For example, a search query like *"car engine issue"* may not return results containing *"vehicle motor problem"*, even though both have similar meanings.

Another major limitation of existing systems is their dependency on internet connectivity. Many modern AI-powered search solutions rely on cloud-based models, which require continuous internet access to function. This not only increases response time but also raises concerns related to data privacy and security.

Additionally, large AI-based systems often require high computational resources, large storage capacity, and significant processing time, making them unsuitable for lightweight or real-time applications.

Therefore, the current systems are limited in terms of:

- Understanding user intent and context
- Providing accurate semantic results
- Ensuring user privacy
- Operating efficiently in offline environments
- Delivering fast and real-time performance

2.1 Survey of Existing Systems

SNo	Existing System	Description	Advantages	Limitations
1	Traditional Keyword based search engines	Systems like browser search (Ctrl+F) or basic search engines that match exact keywords in documents or webpages.	Simple to use, fast for exact matches, widely available.	Cannot understand context or meaning; fails with synonyms or different phrasing.
2	Online Search Engines (Google, Bing)	Web-based search engines that use advanced algorithms and AI to retrieve relevant information from the internet.	Provides vast information, intelligent ranking, and quick results.	Requires internet connection; raises privacy concerns due to data tracking.
3	Cloud-Based AI Search Systems	AI-powered platforms that use large models to perform semantic search through APIs.	High accuracy, understands context and meaning effectively.	It depends on cloud services, high latency, data privacy risks, and costly infrastructure.
4	PDF Search Tools	Built-in PDF readers or tools that allow searching text inside documents using keywords.	Easy document navigation, fast keyword lookup.	Limited to exact word matching; cannot interpret semantic meaning.

5	Local Desktop Search Tools	Applications that search files stored locally on a system using indexing techniques.	Works offline, faster for local data retrieval.	Mostly keyword-based; lacks semantic understanding and advanced AI capabilities.

Analysis:

- Most systems depend on internet connectivity.
- Large AI models consume high memory.
- Privacy concerns due to data upload.

3. PROBLEM STATEMENT

In today's digital world, users deal with large amounts of data such as webpages, PDFs, and documents. Finding relevant information quickly from this data is a major challenge.

Most existing search systems use **keyword-based searching**, which matches exact words from the user query with the content. These systems do not understand the **meaning or context** of the query. As a result, they often fail to return relevant results when different words are used for the same concept.

For example, a search for *"network issue"* may not show results containing *"connection problem"*, even though both have similar meanings. This reduces the efficiency of traditional search systems.
Modern semantic search systems solve this problem by understanding context using Artificial Intelligence.

However, they have several drawbacks:

- Require internet connection
- Depend on cloud services
- Raise privacy concerns
- Use large and resource-heavy models

Additionally, browser tools like Ctrl + F are limited to simple keyword matching and cannot handle meaning-based search.

Therefore, the main problem is:

To develop a fast, lightweight, and offline semantic search system that can understand user intent and provide accurate results without relying on internet or cloud services.

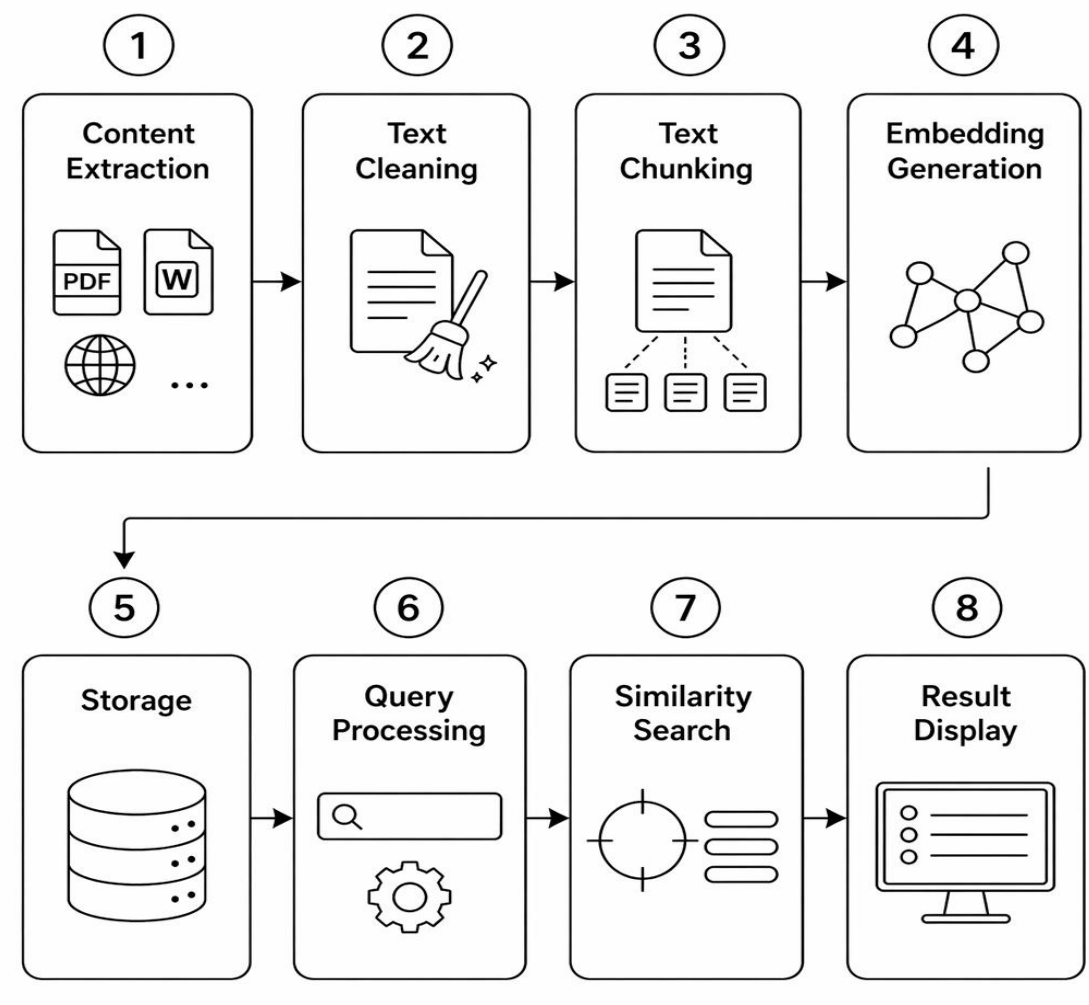
This project aims to provide a solution that ensures speed, accuracy, and data privacy in search operations.

4. PROPOSED METHODOLOGY

The proposed system aims to develop a **high-speed offline semantic search browser extension** that can understand the meaning of user queries and retrieve relevant information from webpages, PDFs, and documents without using internet services.

The methodology is divided into multiple modules and processing steps that work together to achieve efficient semantic search.

4.1 Overall System Workflow



4.2 Module Description

1. User Interface (UI) Module:
This module provides interaction between the user and the system.

Functions:
- Accept search query
- Display results
- Provide indexing option

2. Controller Module:
This acts as the **central control unit** of the system.

Functions:
- Manages system workflow
- Coordinates all modules
- Handles search requests

3. Content Extraction Module:
This module extracts text data from different sources.

Sources:
- Webpages
- PDF documents

Working:
- Reads visible content from the browser
- Sends extracted text for processing

4. Text Processing Module:
Before analysis, the extracted text must be cleaned and prepared.

Steps:
- Remove HTML tags and unwanted symbols
- Normalize text (spacing, encoding)

This improves accuracy and efficiency.

5. Text Chunking Module:
Large text is divided into smaller parts called **chunks**.

Reason:

- AI models cannot process large text at once
- Improves processing speed

Typical Size:

- 200–500 words per chunk

6. Embedding Generation Module:
This is the **core module** of the system.

Working:

- Converts text into numerical vectors (embeddings)
- Uses a lightweight AI model

Example:

- "Machine learning is useful" → Vector form

These embeddings represent the **semantic meaning** of the text.

7. Storage and Indexing Module:
All embeddings are stored locally for fast retrieval.

Technology Used:

- Indexed DB

Stored Data:

- Text chunks
- Vector embeddings
- Metadata

Advantages:

- Avoids recomputation
- Improves search speed

8. Query Processing Module:
When a user enters a query:

Steps:

- Convert query into embedding
- Prepare it for comparison

9. Similarity Search Module:
This module finds relevant results.

Working:

- Compares query vector with stored vectors
- Uses cosine similarity

10. Result Display Module:
Displays the final output to the user.

Features:

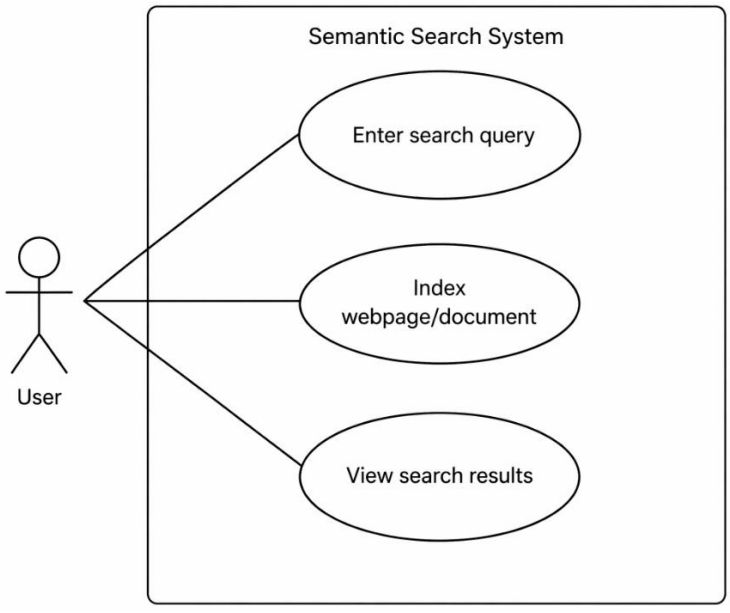
- Ranked results
- Highlighted text
- Quick navigation

4.3 Use Case

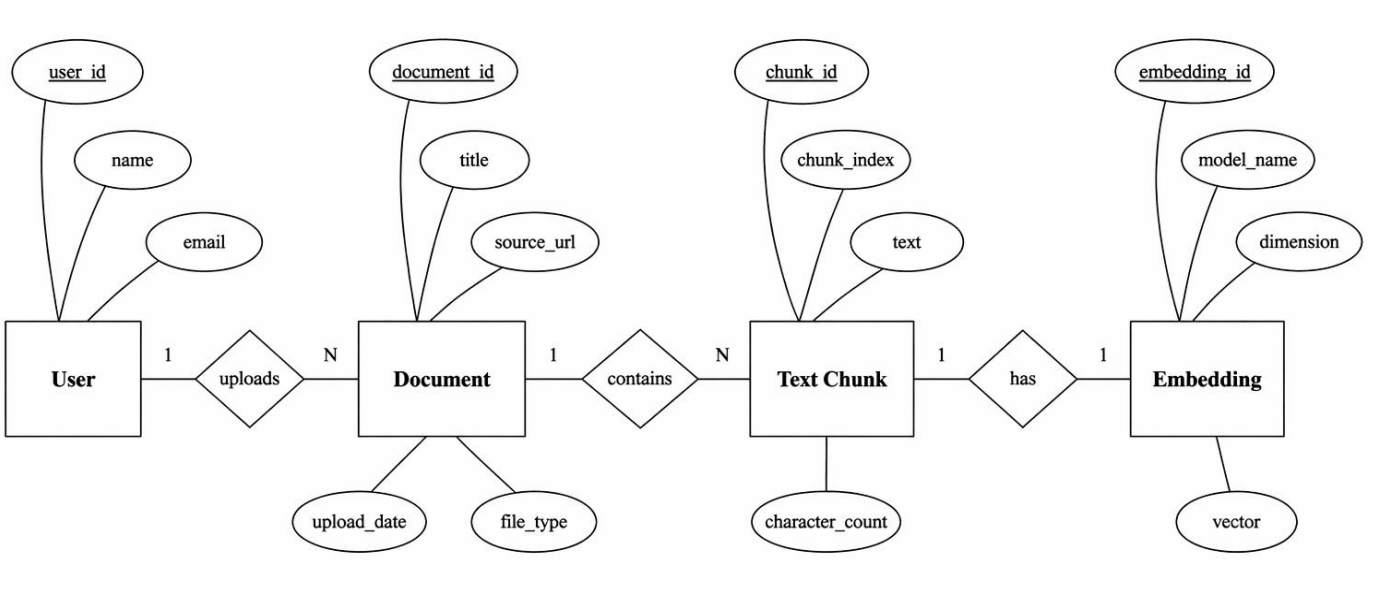
Actor: User
Use Cases:

- Enter search query
- Index webpage/document
- View search results

Diagram:

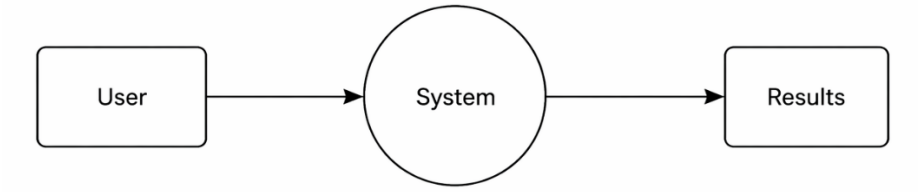


4.4 ER Diagram

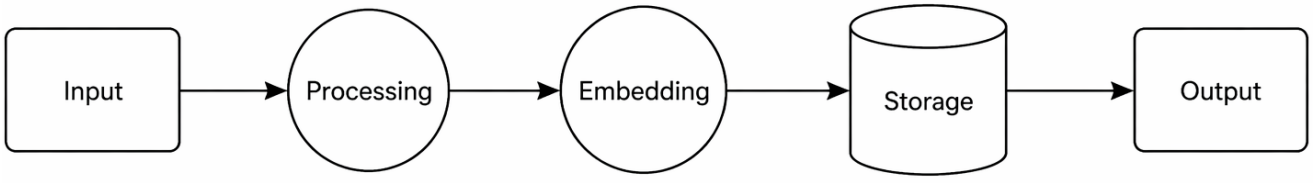


4.5 Data Flow Diagram (DFD)

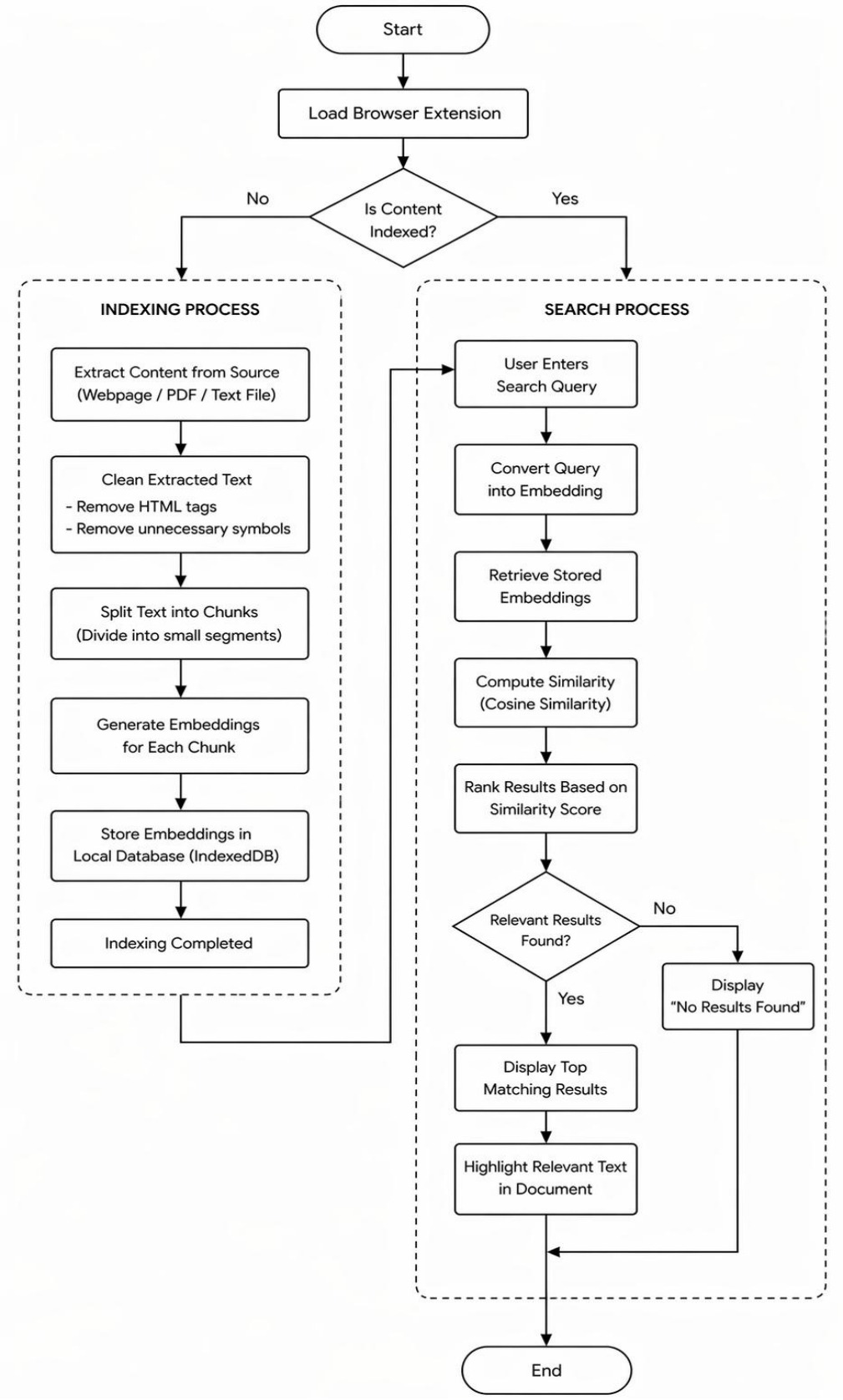
Level 0



Level 1



4.6 Flowchart



5. FEASIBILITY STUDY

Feasibility study is the first and most important step in software development. It is used to determine whether the proposed system is practical, achievable, and beneficial. It analyses different aspects such as technical capability, cost, usability, and overall impact of the system.

The proposed project, **Offline Semantic Search Browser Extension**, is evaluated on the following feasibility factors.

5.1 Need of the Project

In the current scenario, most search systems either rely on keyword matching or require internet-based AI services. These approaches have several limitations such as lack of semantic understanding, dependency on cloud services, and privacy concerns.

There is a strong need for a system that:

- Understands the meaning of user queries
- Works without internet connection
- Provides fast and accurate results
- Ensures user data privacy

The proposed system fulfils these requirements by providing offline semantic search with a lightweight AI model.

5.2 Technical Feasibility

Technical feasibility determines whether the required technology and resources are available to implement the system.

The proposed system is technically feasible because:

- Uses modern web technologies like JavaScript, TypeScript, and Browser Extensions
- Uses lightweight AI models that can run locally (~100 MB)
- Supports ONNX Runtime Web for in-browser AI processing

- Uses Indexed DB for local storage
- Can run on standard computers without high-end hardware

Therefore, the system can be developed and executed efficiently using available tools and technologies.

5.3 Operational Feasibility

Operational feasibility checks whether the system will be easy to use and accepted by users.

- Simple and user-friendly interface
- Works offline, making it convenient

Users can easily perform semantic search without complexity, making the system highly practical.

5.4 Legal and Ethical Feasibility

This ensures that the system follows legal and ethical standards.

- No user data is sent to external servers
- Fully privacy-focused system
- No violation of data protection rules
- Works within browser permissions

Thus, the system is safe and ethically acceptable.

5.6 Conclusion of Feasibility Study

Based on the above analysis, the proposed system is:

- Technically possible
- Economically viable
- Operationally practical
- Legally safe

Hence, the project is feasible and suitable for implementation.

6. FACILITIES REQUIRED FOR PROPOSED WORK

The development of the **Offline Semantic Search Browser Extension** requires basic hardware and software resources for smooth implementation and testing.

Hardware Requirements:

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: 10 GB free space
- System: 64-bit computer

Software Requirements:

- Operating System: Windows / Linux / macOS
- Code Editor: Visual Studio Code
- Runtime: Node.js
- Browser: Chrome or Edge

Programming & Tools:

- Languages: JavaScript, HTML, CSS
- Frameworks: React / Svelte
- AI Tool: ONNX Runtime Web
- Database: Indexed DB
- PDF Tool: pdf.js

Network Requirement:

- Internet required only for setup and installation
- System works offline after setup

The required facilities are simple and easily available. The project does not require expensive resources, making it cost-effective and easy to implement.

7. CONCLUSION

The proposed project, **Offline Semantic Search Browser Extension**, provides an effective solution to the limitations of existing search systems. Traditional keyword-based search fails to understand the meaning of queries, while modern AI-based systems depend on internet connectivity and raise privacy concerns.

This project introduces a lightweight and offline semantic search system that works directly inside the browser. By converting text and queries into vector embeddings, the system is able to match results based on meaning rather than exact words, improving accuracy and relevance.

The system ensures fast performance by using local storage and efficient indexing, achieving quick response times without relying on external servers. It also maintains data privacy, as all processing is done locally on the user's device.

Overall, the project successfully demonstrates that semantic search can be implemented in a simple, efficient, and privacy-friendly manner using modern web technologies.

In future, the system can be enhanced by:

- Supporting larger datasets with optimized indexing techniques
- Improving model accuracy with better embeddings
- Adding voice-based search features
- Integrating multi-language support

Thus, the project provides a practical, efficient, and innovative solution for modern search requirements, ensuring speed, accuracy, and privacy.

REFERENCES

[1] T. Reimers, I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, 2019.

[2] J. Devlin, M. Chang, K. Lee, K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", NAACL-HLT, 2019.

[3] OpenAI, "Embedding Models for Semantic Search", OpenAI Documentation, 2023.

[4] Microsoft, "ONNX Runtime: Cross-platform, High Performance ML Inference and Training Accelerator", Microsoft Documentation, 2022.

[5] Mozilla, "IndexedDB API", MDN Web Docs, 2023.

[6] Mozilla, "Browser Extensions (WebExtensions API)", MDN Web Docs, 2023.

[7] A. Malkov, D. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs", IEEE Transactions on Pattern Analysis and Machine Intelligence, 2018.

[8] Mozilla, "pdf.js – JavaScript PDF Rendering Library", Mozilla Project Documentation, 2023.